



POLITIQUE DE SÉCURITÉ

Cours 2



Dr. Sarra Namane

Département d'informatique

Université Badji Mokhtar, Annaba

Promotion: Master 1 (RSI)

Année: 2023/2024

Introduction au modèle de contrôle d'accès

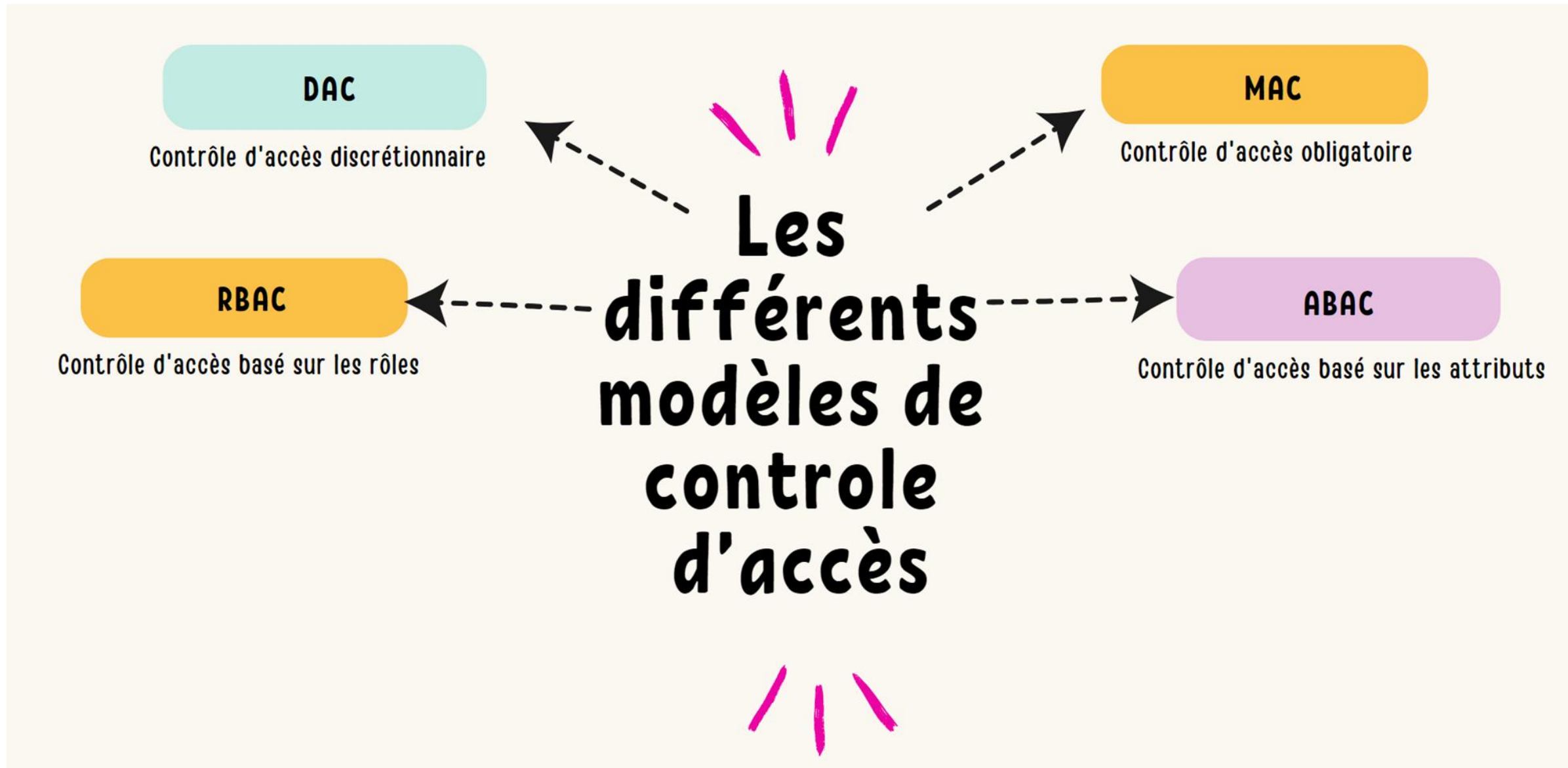
- Dans le domaine de l'informatique et des technologies de l'information, la sécurité joue un rôle crucial pour protéger les systèmes, les données et les utilisateurs contre les menaces potentielles.
- Les **modèles de sécurité** fournissent un cadre conceptuel essentiel pour comprendre, analyser et mettre en œuvre des mesures de sécurité efficaces.
- Les **modèles de sécurité** définissent les principes et les objectifs fondamentaux de la sécurité informatique, tels que la confidentialité, l'intégrité et la disponibilité des données. Ils fournissent un cadre conceptuel pour comprendre les menaces potentielles et élaborer des politiques de sécurité appropriées. **Les modèles de contrôle d'accès**, quant à eux, font partie intégrante des modèles de sécurité et se concentrent spécifiquement sur la gestion de l'accès aux ressources informatiques.



Définition d'un modèle de contrôle d'accès

- Un modèle de contrôle d'accès est un **cadre conceptuel formel** qui permet de décrire précisément le fonctionnement d'un système afin de le comprendre de manière approfondie.
- Il fournit une **représentation abstraite** du système, ce qui simplifie la compréhension de la politique de sécurité qui régit l'accès aux ressources.
- En utilisant ce modèle, les concepteurs peuvent spécifier les règles et les politiques de contrôle d'accès de manière claire et précise.
- De plus, le modèle facilite l'implémentation des mécanismes techniques nécessaires pour appliquer ces politiques et atteindre les objectifs de sécurité définis.
- En résumé, un modèle de contrôle d'accès permet de formaliser et de simplifier la gestion de l'accès aux ressources informatiques, tout en assurant la sécurité du système.

Les différents modèles de contrôle d'accès



Le modèle DAC

Définition 1

- La définition du contrôle d'accès discrétionnaire dans TCSEC (Trusted Computer System Evaluation Criteria) stipule que ce mécanisme permet de restreindre l'accès aux objets en fonction de l'identité des utilisateurs ou des groupes auxquels ils appartiennent. Ce contrôle est qualifié de "discrétionnaire" car il donne au sujet la capacité de transférer les autorisations d'accès à d'autres utilisateurs.
- En d'autres termes, le propriétaire de l'objet peut choisir de partager l'accès à cet objet avec d'autres utilisateurs, et cette décision est laissée à sa discrétion. Cela signifie que le sujet a le pouvoir discrétionnaire de déléguer les droits d'accès à d'autres utilisateurs selon ses propres critères et besoins.

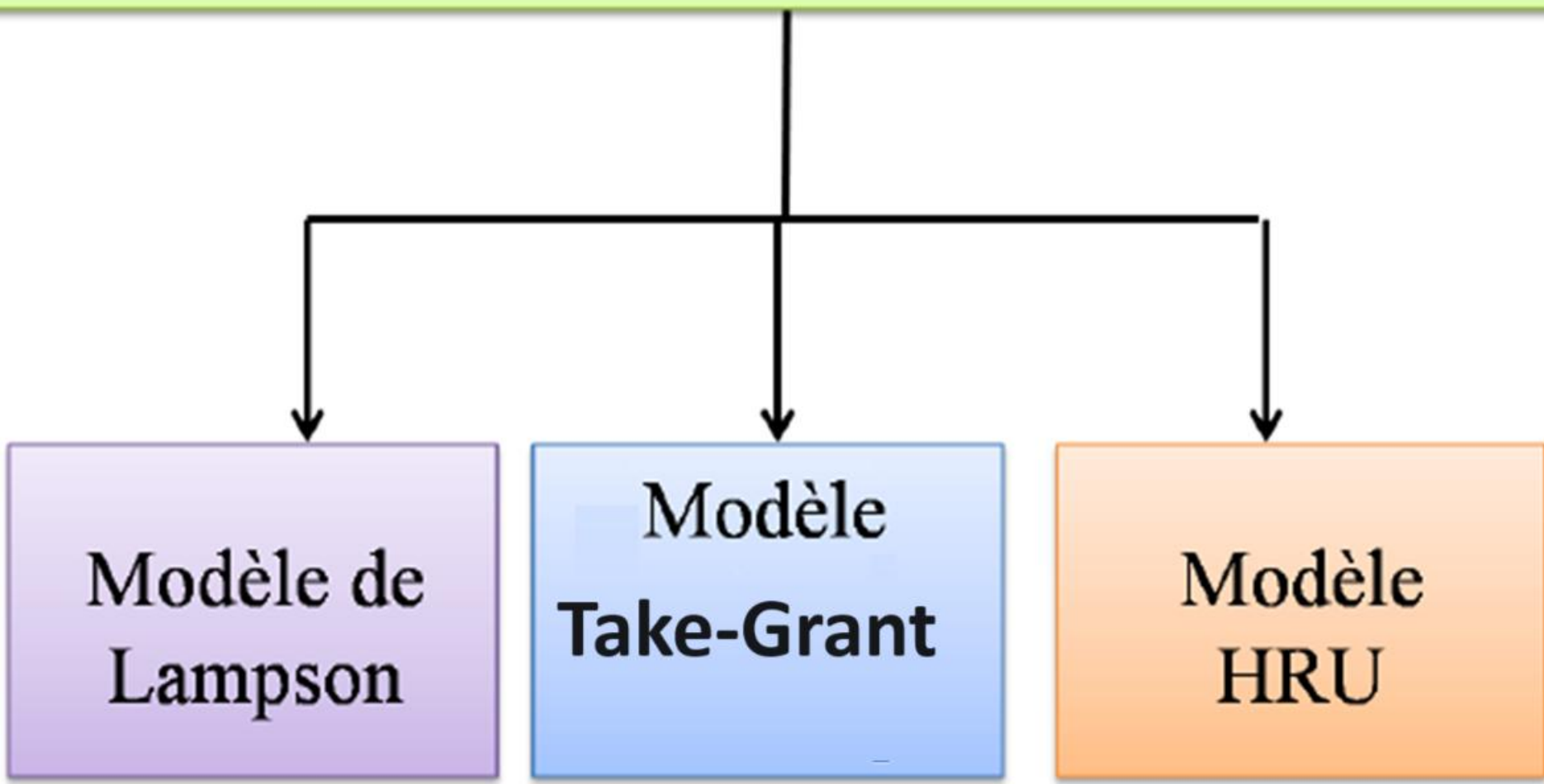
Le modèle DAC (2)

Définition 2

- Dans la deuxième définition, le contrôle d'accès est qualifié de "discrétionnaire" car il repose sur l'identité du sujet, c'est-à-dire l'utilisateur ou le processus qui demande l'accès à un objet. Dans ce contexte, un "sujet" peut être un utilisateur individuel ou un groupe d'utilisateurs.
- La caractéristique principale du contrôle d'accès discrétionnaire est que les droits d'accès sont déterminés par le propriétaire ou le créateur de l'objet. Ce propriétaire a la discrétion de spécifier quels utilisateurs ou groupes d'utilisateurs ont accès à cet objet et quel type d'accès leur est accordé (lecture, écriture, exécution, etc.).
- De plus, dans un système de contrôle d'accès discrétionnaire, le sujet qui détient les droits d'accès à un objet a la faculté de transférer ces droits à d'autres utilisateurs, selon ses propres critères. Cela signifie que le propriétaire peut décider de partager l'accès à un objet avec d'autres utilisateurs en leur accordant des autorisations spécifiques.

Les différents modèles DAC

Modèle de contrôle d'accès discrétionnaire (DAC)



```
graph TD; A[Modèle de contrôle d'accès discrétionnaire (DAC)] --> B[Modèle de Lampson]; A --> C[Modèle Take-Grant]; A --> D[Modèle HRU];
```

Modèle de
Lampson

Modèle
Take-Grant

Modèle
HRU

Le modèle de Lampson

Le modèle de Lampson pour le contrôle d'accès, introduit en 1971, est basé sur la notion de droits d'accès spécifiés par une matrice de contrôle d'accès. Ce modèle repose sur trois composantes principales :

1. **Ensemble d'objets (O)** : Il s'agit des entités dans le système qui doivent être protégées. Ces objets peuvent être des fichiers, des répertoires, des dispositifs ou d'autres ressources informatiques.
2. **Ensemble de sujets (S)** : Ce sont les entités qui ont accès aux objets. Les sujets peuvent être des utilisateurs, des processus ou d'autres entités qui interagissent avec les objets du système.
3. **Matrice d'accès (M)** : C'est une structure de données qui spécifie les droits d'accès des sujets aux objets. Chaque entrée de la matrice représente une paire sujet-objet et indique les droits que le sujet possède sur l'objet correspondant. Par exemple, une entrée de la forme $M[s, o] = \{\text{lire, écrire}\}$ signifie que le sujet s a le droit de lire et d'écrire sur l'objet o .

En outre, l'ensemble R représente un ensemble de droits, tels que lire, écrire, exécuter, etc., qui peuvent être attribués aux sujets pour interagir avec les objets du système. Chaque droit r est défini comme une paire sujet-objet, notée $r \in [S, O]$, ce qui signifie qu'il est associé à une relation spécifique entre un sujet et un objet.

En résumé, le modèle de Lampson pour le contrôle d'accès organise les interactions entre les sujets et les objets à l'aide d'une matrice d'accès, spécifiant les droits d'accès des sujets aux objets, et cela est crucial pour assurer la sécurité et la protection des ressources du système.

Exemple d'une matrice de contrôle d'accès

E : écrire, L : lire

	Enregistrement médical	Enregistrement administratif	Prescriptions
Imene	E, L	E	L
Karim		E	L
Mohamed		L	
Nadine	E, L	L	E, L

Le modèle de Lampson (2)

Ces deux concepts, la liste de contrôle d'accès (ACL) et la liste de capacités (ou capability), sont deux approches différentes pour stocker et organiser les autorisations d'accès dans un système de contrôle d'accès.

1. Liste de contrôle d'accès (ACL) :

Dans ce modèle, la matrice d'accès est stockée par colonne, ce qui signifie que pour chaque objet, il existe une liste d'autorisations associée à chaque sujet. Chaque colonne de la matrice représente un objet et contient une liste des droits accordés à chaque sujet pour cet objet spécifique. Par exemple, pour un fichier donné, la colonne correspondante de la matrice ACL contiendrait une liste des utilisateurs autorisés et les actions qu'ils peuvent effectuer sur ce fichier.

2. Liste de capacités (ou capability) :

Dans ce modèle, la matrice d'accès est stockée par ligne, ce qui signifie que pour chaque sujet, il existe une liste d'autorisations associée à chaque objet. Chaque ligne de la matrice représente un sujet et contient une liste des droits que ce sujet possède sur chaque objet du système. Par exemple, pour un utilisateur donné, la ligne correspondante de la matrice de capacité contiendrait une liste des objets auxquels il peut accéder et les actions qu'il peut effectuer sur chacun d'eux.

En résumé, la différence principale entre l'ACL et la capacité réside dans la manière dont les autorisations sont stockées et organisées : l'ACL stocke les autorisations par objet, tandis que la capacité stocke les autorisations par sujet. Ces deux approches ont leurs avantages et inconvénients, et le choix entre les deux dépend des besoins spécifiques du système et des préférences de conception.

Exercice sur le modèle Lampson

Hypothèse :

Considérons trois utilisateurs : David, Emma et Félix.

Nous avons trois fichiers : rapport-financier, présentation_projet et feuille_calcul, appartenant respectivement à David, Emma et Félix.

De plus :

- David peut lire et écrire dans présentation_projet.
- Emma peut lire rapport_financier et écrire dans feuille_calcul.
- Félix peut lire et exécuter rapport_financier, lire et écrire dans présentation_projet et lire sur feuille_calcul.
- Chaque propriétaire peut exécuter les fichiers qui lui appartiennent.

Questions :

1. Représentez la matrice d'accès.
2. Effectuez les actions suivantes et représentez la nouvelle matrice d'accès :
 - Félix donne à Emma le droit de lire et d'écrire dans rapport_financier.
 - David retire le droit de lecture à Félix sur rapport _ financier.

Solution

	rapport_financier	présentation_projet	feuille_calcul
David	X	L,E	
Emma	L	X	E
Félix	L,X	L,E	L,X

L: Lire
E: Ecrire
X: Exécuter

Nouvelle matrice

	rapport_financier	présentation_projet	feuille_calcul
David	X	L,E	
Emma	L, E	X	E
Félix	X	L,E	L,X

Modèle de Harrison-Ruzzo-Ullman

Le modèle de Harrison-Ruzzo-Ullman (HRU) est une amélioration du modèle de contrôle d'accès discrétionnaire de Lampson. Dans ce modèle, une matrice de contrôle d'accès classique est utilisée, mais ce qui le distingue est que **HRU précise les commandes qui peuvent être appliquées à chaque objet.**

Contrairement au modèle de **Lampson**, où les autorisations sont simplement définies par des droits d'accès (lecture, écriture, etc.), le modèle HRU introduit une distinction entre les droits d'accès et les commandes. Les droits d'accès déterminent quelles actions un sujet peut effectuer sur un objet (par exemple, lire ou écrire), tandis que les commandes spécifient les opérations spécifiques qui peuvent être effectuées sur un objet (par exemple, copier, supprimer, renommer).

Ainsi, avec le modèle HRU, la matrice de contrôle d'accès spécifie non seulement les droits d'accès des sujets aux objets, mais elle précise également les commandes qui peuvent être appliquées à chaque objet. Cette distinction permet une granularité plus fine dans la gestion des autorisations et offre un meilleur contrôle sur les actions pouvant être effectuées dans le système.

Modèle de Harrison-Ruzzo-Ullman (2)

Les états de protection

Les états de protection sont un concept fondamental dans les modèles de contrôle d'accès. Voici une explication détaillée des composants et des aspects associés aux états de protection:

- **SC (Sujets)**: Il s'agit d'un ensemble dénombrable de constantes représentant les sujets du système. Chaque élément de cet ensemble est noté s ou s' .
- **OC (Objets)**: C'est également un ensemble dénombrable de constantes qui représente les objets du système. Chaque élément de cet ensemble est noté o ou o' .
- **R (Droits)**: Il s'agit d'un ensemble fini de droits, représentant les actions autorisées sur les objets du système. Chaque élément de cet ensemble est noté r ou r' .
- **Relation entre SC et OC**: Dans le contexte des modèles de contrôle d'accès, comme celui de Harrison, Ruzzo et Ullman, tous les sujets sont également considérés comme des objets. Cela est symbolisé par l'inclusion de l'ensemble SC dans l'ensemble OC, ce qui signifie que chaque sujet est également un objet.

Modèle de Harrison-Ruzzo-Ullman (3)

- **État de protection:** Il est représenté par un triplet (S, O, P) , où :
 - S est l'ensemble des sujets,
 - O est l'ensemble des objets,
 - P est une matrice d'accès qui spécifie les droits que chaque sujet a sur chaque objet. Pour chaque sujet s dans S et pour chaque objet o dans O , $P[s, o]$ définit l'ensemble des droits que le sujet s a sur l'objet o .
- Notation des états de protection : Les états de protection sont souvent représentés par les lettres Δ, Δ' .

Modèle de Harrison-Ruzzo-Ullman (4)

Les opérations primitives

Les **opérations primitives** sont des actions de base qui permettent de modifier l'état de protection dans un système de contrôle d'accès. Voici une explication détaillée des composants et des opérations primitives associées :

- **SV (Variables de sujets)**: Il s'agit d'un ensemble dénombrable de variables représentant les sujets du système. Chaque élément de cet ensemble est noté x ou x' .
- **OV (Variables d'objets)**: C'est également un ensemble dénombrable de variables représentant les objets du système. Chaque élément de cet ensemble est noté y ou y' .
- **Opérations primitives** : Il existe six opérations primitives qui permettent de modifier l'état de protection :
 1. **Créer le sujet x** : Cette opération crée un nouveau sujet dans le système, représenté par la variable x .

Modèle de Harrison-Ruzzo-Ullman (5)

- 2. Créer l'objet y :** Cette opération crée un nouvel objet dans le système, représenté par la variable y .
- 3. Introduire r dans $P[x, y]$:** Cette opération ajoute un droit r dans la matrice d'accès pour le sujet x sur l'objet y .
- 4. Détruire le sujet x :** Cette opération supprime le sujet x du système, ainsi que toutes ses associations avec les objets.
- 5. Détruire l'objet y :** Cette opération supprime l'objet y du système, ainsi que toutes les associations avec les sujets.
- 6. Supprimer r de $P[x, y]$:** Cette opération retire le droit r de la matrice d'accès pour le sujet x sur l'objet y .

- Notation des opérations primitives : Les opérations primitives sont souvent représentées par la lettre grecque π .

Modèle de Harrison-Ruzzo-Ullman (6)

Dans le modèle de Harrison-Ruzzo-Ullman (HRU), les opérations primitives sont appliquées indirectement à travers des commandes de la forme :

si C_1 et ... et C_i alors début π_1 ; ... ; π_j

Où $C_1, \dots, C_i$ sont des **conditions élémentaires** de la forme $r \in P[x, y]$ et $\pi_1, \dots, \pi_j$ sont des **opérations primitives**.

- **Conditions élémentaires:** Ce sont des conditions qui spécifient si un certain droit est présent dans la matrice d'accès pour une certaine paire sujet-objet. Par exemple, $r \in P[x, y]$ signifie que le droit r est présent pour le sujet x sur l'objet y .
- **Opérations primitives:** Ce sont les opérations de base qui modifient l'état de protection, comme expliqué précédemment.

Les commandes sont exécutées en remplaçant toutes les variables de la commande par des constantes. Si les conditions initiales $C_1, \dots, C_i$ sont évaluées à vrai dans l'état de protection courant, alors les opérations primitives $\pi_1, \dots, \pi_j$ sont exécutées séquentiellement.

Les commandes dans le modèle HRU sont souvent notées par la lettre grecque α .

Modèle de Harrison-Ruzzo-Ullman (7)

Assignment (θ) et Commande (α) : Dans ce contexte, l'assignation θ représente un ensemble de valeurs attribuées aux variables du système, tandis que la commande α est une instruction conditionnelle de la forme "si C_1 et ... et C_i alors début π_1 ; ... ; π_j fin". Les variables dans α sont remplacées par des constantes en utilisant Δ , ce qui signifie que la commande α est fixée pour une configuration donnée du système.

États de Protection (Δ et Δ') : Dans le modèle HRU, un état de protection est représenté par un triplet (S, O, P) , où S représente l'ensemble des sujets, O l'ensemble des objets et P les autorisations d'accès entre sujets et objets. Ainsi, Δ représente l'état de protection initial et Δ' l'état de protection résultant après l'application de la commande α .

Modèle de Harrison-Ruzzo-Ullman (8)

Dérivation des États de Protection

On dit que l'état de protection Δ' est dérivable de Δ en utilisant θ et α si l'une des deux conditions suivantes est satisfaite :

a. Si les conditions de la commande α ne sont pas satisfaites, alors l'état de protection reste inchangé ($\Delta' = \Delta$). Cela signifie que l'exécution de la commande α n'a pas d'impact sur l'état de protection.

b. Sinon, il existe une séquence d'états de protection $\Delta_0, \Delta_1, \dots, \Delta_j$, où Δ_0 correspond à l'état initial Δ et Δ_j est l'état final Δ' , tels que chaque état Δ_i est obtenu en appliquant les actions π_i de la commande α à l'état précédent Δ_{i-1} . Cela signifie que l'exécution de la commande α entraîne une série de changements dans l'état de protection, conduisant finalement à l'état de protection Δ' .

$$\Delta = \Delta_0 \rightarrow \Delta_1 \rightarrow \Delta_2 \rightarrow \Delta_3 \rightarrow \Delta_4 \rightarrow \dots \rightarrow \Delta_j = \Delta'$$

Modèle de Harrison-Ruzzo-Ullman (9)

Exemple sur Dérivation des États de Protection

Supposons que nous ayons un système informatique avec trois sujets (utilisateurs) : Alice, Bob et Carol, et trois objets (ressources) : Fichier A, Fichier B et Fichier C. Les autorisations d'accès sont les suivantes :

- Alice a accès en lecture au Fichier A.
- Bob a accès en lecture et écriture au Fichier B.
- Carol a accès en lecture au Fichier C.

L'état de protection initial ($\triangle$) serait représenté comme suit :

$\triangle = (S, O, P)$ $S = \{\text{Alice, Bob, Carol}\}$

$O = \{\text{Fichier A, Fichier B, Fichier C}\}$

$P = \{ (\text{Alice, Fichier A, lecture}), (\text{Bob, Fichier B, lecture}), (\text{Bob, Fichier B, écriture}), (\text{Carol, Fichier C, lecture}) \}$

Maintenant, supposons que la commande α soit "**si Alice a accès en lecture au Fichier A, alors supprimer l'accès en lecture de Carol au Fichier C**".

Dans ce cas, la condition de la commande α est **satisfaite** car Alice a effectivement accès en lecture au Fichier A. Ainsi, nous devons modifier l'état de protection pour refléter cela. Cependant, comme Bob n'a pas accès au Fichier C, la deuxième action de la commande α ne sera pas appliquée.

La séquence des états de protection serait la suivante :

1. $\triangle_0 = (S, O, P)$: état initial

2. $\triangle_1$: Alice supprime l'accès en lecture de Carol au Fichier C

- $\triangle_1 = (S, O, P')$ où $\triangle' = \{ (\text{Alice, Fichier A, lecture}), (\text{Bob, Fichier B, lecture}), (\text{Bob, Fichier B, écriture}) \}$

Ainsi, dans cet exemple, l'état de protection $\triangle'$ est dérivable de $\triangle$ en utilisant l'assignation et la commande spécifiées, car les actions correspondantes de la commande α sont appliquées avec succès, entraînant un changement dans l'état de protection initial.

Modèle de Harrison-Ruzzo-Ullman (10)

Transition des états

La transition des états dans le modèle HRU est définie par les actions suivantes :

1. **"Créer le sujet x"**: Si x n'est pas déjà un sujet ($\theta(x)$ n'appartient pas à S), alors l'état transitionnel est défini comme suit :
 1. S' est l'ensemble des sujets après la transition, obtenu en ajoutant $\theta(x)$ à l'ensemble initial S .
 2. O' est l'ensemble des objets après la transition, obtenu en ajoutant $\theta(x)$ à l'ensemble initial O .
 3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) , sauf pour $(\theta(x), o)$ où $P'[\theta(x), o]$ est vide pour tout o dans O' et pour $(s, \theta(x))$ où $P'[s, \theta(x)]$ est vide pour tout s dans S' .
 $(x) \notin S, S' = S \cup \{\theta(x)\}, O' = O \cup \{\theta(x)\}, P'[s, o] = P[s, o] \text{ pour tout } (s, o) \in S \times O, P'[\theta(x), o] = \emptyset;$
2. **"Détruire le sujet x"**: Si $\theta(x)$ est un sujet, alors l'état transitionnel est défini comme suit :
 1. S' est l'ensemble des sujets après la transition, obtenu en supprimant $\theta(x)$ de l'ensemble initial S .
 2. O' est l'ensemble des objets après la transition, obtenu en supprimant $\theta(x)$ de l'ensemble initial O .
 3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S' \times O'$.
 $\theta(x) \in S, S' = S \setminus \{\theta(x)\}, O' = O \setminus \{\theta(x)\} \text{ et } P'[s, o] = P[s, o] \text{ pour tout } (s, o) \in S' \times O'.$
3. **"Créer l'objet y"**: Si y n'est pas déjà un objet ($\theta(y)$ n'appartient pas à O), alors l'état transitionnel est défini comme suit :
 1. S' est conservé tel quel.
 2. O' est l'ensemble des objets après la transition, obtenu en ajoutant $\theta(y)$ à l'ensemble initial O .
 3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S \times O$, sauf pour $(s, \theta(y))$ où $P'[s, \theta(y)]$ est vide pour tout s dans S' .
 $\theta(y) \notin O, S' = S, O' = O \cup \{\theta(y)\}, P'[s, o] = P[s, o] \text{ pour tout } (s, o) \in S \times O \text{ et } P'[s, \theta(y)] = \emptyset; \text{ pour tout } s \in S'.$

Modèle de Harrison-Ruzzo-Ullman (11)

Transition des états (2)

4. "**Détruire l'objet y** ": Si $\theta(y)$ est un objet qui n'est pas associé à un sujet, alors l'état transitionnel est défini comme suit :

1. S' est conservé tel quel.
2. O' est l'ensemble des objets après la transition, obtenu en supprimant $\theta(y)$ de l'ensemble initial O .
3. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S' \times O'$.

$\theta(y) \in O \setminus S, S' = S, O' = O \setminus \{\theta(y)\}$ et $P'[s, o] = P[s, o]$ pour tout $(s, o) \in S' \times O'$.

5. "**Introduire r dans $P(x, y)$** ": Si $\theta(x)$ est un sujet et $\theta(y)$ est un objet, alors l'état transitionnel est défini comme suit :

1. S' et O' sont conservés tels qu'ils sont.
2. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S \times O$, sauf pour $(\theta(x), \theta(y))$ où r est ajouté à la liste des permissions.

$\theta(x) \in S, \theta(y) \in O, S' = S, O' = O$ et pour tout $(s, o) \in S \times O$: $P'[s, o] = \begin{cases} P[s, o] \cup \{r\} & \text{si } s = \theta(x) \text{ et } o = \theta(y) \\ P[s, o] & \text{sinon} \end{cases}$

6. "**Supprimer r de $P(x, y)$** ": Si $\theta(x)$ est un sujet et $\theta(y)$ est un objet, alors l'état transitionnel est défini comme suit

1. S' et O' sont conservés tels qu'ils sont.
2. La matrice P' conserve les mêmes valeurs que P pour tous les états et actions (s, o) dans $S \times O$, sauf pour $(\theta(x), \theta(y))$ où r est retiré de la liste des permissions.

$\theta(x) \in S, \theta(y) \in O, S' = S, O' = O$ et pour tout $(s, o) \in S \times O$: $P'[s, o] = \begin{cases} P[s, o] \setminus \{r\} & \text{si } s = \theta(x) \text{ et } o = \theta(y) \\ P[s, o] & \text{sinon} \end{cases}$

Modèle de Harrison-Ruzzo-Ullman (12)

Exemple de commande

- **Création de fichier :**

Command créer-fichier(p,f)

créer_objet(f)

entrer(own,M[p,f])

entrer (r,M[p,f])

entrer (w,M[p,f])

Fin

- **Donner un accès en lecture sur un fichier :**

Command grand-read(p,f,q)

Si

own $\in$ M[p,f]

Alors

entrer(r,M[q,f])

Fin

**Commande qui
indique la c création
d'un fichier f par le
processus P**

**Commande qui permet à
un sujet p propriétaire du
fichier
f d'accorder un accès en
lecture à un autre sujet
q sur ce fichier**

Modèle de Harrison-Ruzzo-Ullman (13)

Exercice

Objets utilisateurs	Fichier1	Fichier 2	Fichier 3	Fichier 4
Ramzi	owns	read	read	
Riad		owns	read, write	owns
Billel	read	read ,write	owns	Read, Write
Nedjma			Read	Read, write

- Définir une commande confer.read qui permet à l'utilisateur Ramzi de donner un accès en lecture sur un fichier 1 à l'utilisateur Riad.

Modèle de Harrison-Ruzzo-Ullman (14)

L'automate d'accessibilité global

si le nombre de sujets et objets créés reste fini, il est possible d'explorer toutes les évolutions possibles du système .

- Cela signifie que dans un système où il y a un nombre limité de sujets (utilisateurs) et d'objets (ressources), il est théoriquement possible d'analyser toutes les combinaisons d'actions possibles que ces sujets peuvent effectuer sur ces objets. Cela permet de déterminer l'état du système à tout moment et de prévoir toutes les évolutions possibles.
- **L'automate d'accessibilité global** est un moyen de représenter toutes les combinaisons d'actions possibles dans un système. Dans cet exemple, l'automate d'accessibilité global montrerait tous les états possibles du système après l'exécution de chaque action. Cela permettrait de visualiser toutes les interactions entre les sujets et les objets et de déterminer les autorisations accordées ou révoquées à chaque étape. Cela aiderait également à identifier les éventuelles incohérences ou violations de sécurité dans le système.

Modèle de Harrison-Ruzzo-Ullman (15)

Exemple

Dans l'exemple donné, le système est initialement constitué de deux sujets, A et B, et aucun objet n'est encore créé. Les étapes suivantes décrivent les actions effectuées par ces sujets sur les objets :

1. A crée un objet F1
2. B crée un objet F2
3. A autorise A à écrire dans F1
4. A autorise B à lire dans F1
5. B autorise A à lire F2
6. Si A a le droit de lire F2, B lui retire ce droit
7. Si B a le droit de lire F1, A lui retire ce droit

Modèle de Harrison-Ruzzo-Ullman (16)

L'automate d'accessibilité global de l'exemple

